

PostgreSQL Live Demo

Quick Start & Cheat Sheet

For the session: *The 10 Most Powerful Features of PostgreSQL*

Presenter: Kellyn Gorman

1) Prerequisites

- PostgreSQL **15–17** installed, psql available on PATH.
- DBeaver can be used or another UI, if preferred.
- (Optional) Superuser or a role with privileges to CREATE DATABASE, CREATE EXTENSION.
- Recommended extensions:
 - postgres_fdw (ships with core)
 - vector (pgvector)
 - citext (for case-insensitive emails)

-----Start Demo after slide #5

Create demo databases

```
-- From psql as a superuser: psql -U postgres
CREATE DATABASE pg_top10_demo; -- main demo DB
CREATE DATABASE ext_source;    -- FDW target DB
```

2) Environment Setup (psql)

```
-- Connect to main DB
\c pg_top10_demo

-- Helpful session settings for consistent plans and readable output
SET client_min_messages = NOTICE;
SET search_path = public;
SET work_mem = '128MB';
SET maintenance_work_mem = '512MB';
SET max_parallel_workers_per_gather = 4;
\timing on
```

Tip: Use two psql windows: one connected to pg_top10_demo (main) and another to ext_source (FDW source).

3) Seed the FDW Source

Connect to ext_source and create some data to query remotely.

```
\c ext_source
CREATE TABLE IF NOT EXISTS products (
  id SERIAL PRIMARY KEY,
  sku TEXT UNIQUE,
  attrs JSONB
);
INSERT INTO products (sku, attrs) VALUES
('SKU-001', '{"category": "apparel", "size": "M"}')
ON CONFLICT DO NOTHING;
INSERT INTO products (sku, attrs) VALUES
('SKU-002', '{"category": "hardware", "size": "N/A"}')
ON CONFLICT DO NOTHING;
```

4) Run the Demos (Main DB)

Main DB Demo- Hybrid data and GIN demonstration, this is the one demo that is out of order from the slides, as the indexing is farther into the presentation.

A) JSON/JSONB (hybrid data + GIN)

```
\c pg_top10_demo
DO $$ BEGIN CREATE EXTENSION IF NOT EXISTS citext; END $$;
DROP TABLE IF EXISTS customers CASCADE;
CREATE TABLE customers (
  customer_id SERIAL PRIMARY KEY,
  name TEXT NOT NULL,
  email CITEXT,
  created_at TIMESTAMPTZ DEFAULT now(),
  profile JSONB
);
INSERT INTO customers (name, email, profile) VALUES
('Ada Lovelace', 'ada@example.com',
 '{"tier": "gold", "prefs": {"lang": "en", "theme": "dark"}, "tags": ["math", "ai"]}' ),
('Grace Hopper', 'grace@example.com', '{"tier": "silver", "prefs": {"lang": "en"}, "tags": ["compiler"]}' ),
('Katherine Johnson', 'kj@example.com', '{"tier": "platinum", "prefs": {"lang": "en", "theme": "light"}, "tags": ["nasa", "orbital"]}' );
CREATE INDEX IF NOT EXISTS customers_profile_gin ON customers USING GIN
(profile jsonb_path_ops);
-- Filter on JSON keys
```

```

EXPLAIN (ANALYZE, BUFFERS)
SELECT customer_id, name
FROM customers
WHERE profile ->> 'tier' IN ('gold','platinum');
-- Nested attribute match
EXPLAIN (ANALYZE, BUFFERS)
SELECT name, profile->'prefs'->>'theme' AS theme
FROM customers
WHERE profile @> '{"prefs":{"lang":"en"}}';

```

-----First Demo Section Complete

-----Start after Slide #7

B) FDW (unify external data with postgres_fdw)

```

DO $$ BEGIN CREATE EXTENSION IF NOT EXISTS postgres_fdw; END $$;
DROP SERVER IF EXISTS ext_source_svr CASCADE;
CREATE SERVER ext_source_svr FOREIGN DATA WRAPPER postgres_fdw
OPTIONS (host '127.0.0.1', dbname 'ext_source', port '5432');
DROP USER MAPPING IF EXISTS FOR CURRENT_USER SERVER ext_source_svr;
CREATE USER MAPPING FOR CURRENT_USER SERVER ext_source_svr OPTIONS (user
'postgres', password 'password');
DROP FOREIGN TABLE IF EXISTS ext_products;
CREATE FOREIGN TABLE ext_products (
    id    INTEGER,
    sku   TEXT,
    attrs JSONB
) SERVER ext_source_svr OPTIONS (table_name 'products');
EXPLAIN (ANALYZE, VERBOSE)
SELECT c.name, p.sku, p.attrs->>'category' AS category
FROM customers c
CROSS JOIN ext_products p
WHERE (c.profile->>'tier') IN ('gold','platinum');

```

-----Second Demo Complete

-----Start after Slide #10

C) Partitioning + Pruning (range by month)

```

DROP TABLE IF EXISTS orders CASCADE;
CREATE TABLE orders (
    order_id    BIGSERIAL NOT NULL,

```

```

    customer_id INT REFERENCES customers(customer_id),
    order_date DATE NOT NULL,
    amount NUMERIC(12,2) NOT NULL,
    status TEXT NOT NULL DEFAULT 'new'
) PARTITION BY RANGE (order_date);
CREATE TABLE orders_2025_01 PARTITION OF orders FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');
CREATE TABLE orders_2025_02 PARTITION OF orders FOR VALUES FROM ('2025-02-01') TO ('2025-03-01');
CREATE TABLE orders_2025_03 PARTITION OF orders FOR VALUES FROM ('2025-03-01') TO ('2025-04-01');
INSERT INTO orders (customer_id, order_date, amount, status)
SELECT (1 + (random()*2)::int), (DATE '2025-01-01' + (random()*89)::int),
      (10 + random()*1000)::numeric(12,2), CASE WHEN random() < 0.8 THEN
'paid' ELSE 'new' END
FROM generate_series(1, 20000) g;
EXPLAIN (ANALYZE, BUFFERS) SELECT sum(amount) FROM orders; -- Likely multi-
partition scan
EXPLAIN (ANALYZE, BUFFERS)
SELECT sum(amount) FROM orders
WHERE order_date >= '2025-02-01' AND order_date < '2025-03-01'; -- pruned

```

-----Completion of Demo

-----Start After Slide #12

D) Parallel Query

```

SET max_parallel_workers_per_gather = 4;
EXPLAIN (ANALYZE, VERBOSE)
SELECT date_trunc('day', order_date) AS d, count(*) AS orders, sum(amount) AS
revenue
FROM orders
GROUP BY 1
ORDER BY 1;

```

----Completion of Demo

-----Start after slide #16

E) Row-Level Security (RLS)

```

DROP TABLE IF EXISTS invoices CASCADE;
CREATE TABLE invoices (

```

```

    invoice_id BIGSERIAL PRIMARY KEY,
    tenant_id TEXT NOT NULL,
    customer_id INT REFERENCES customers(customer_id),
    amount NUMERIC(12,2) NOT NULL,
    created_at TIMESTAMPTZ DEFAULT now()
);
INSERT INTO invoices(tenant_id, customer_id, amount) VALUES
('tenant_a', 1, 199.99), ('tenant_a', 2, 49.00), ('tenant_b', 3, 999.00);

SELECT invoice_id, tenant_id, amount FROM invoices; -- all rows

TESTING for BUG----

create role rls_user NOINHERIT;
GRANT USAGE ON SCHEMA public TO rls_user;

GRANT SELECT, UPDATE, INSERT ON ALL TABLES IN SCHEMA public TO rls_user;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO rls_user;
---End of bug test

ALTER TABLE invoices ENABLE ROW LEVEL SECURITY;
DROP POLICY IF EXISTS tenant_isolation ON invoices;
CREATE POLICY tenant_isolation ON invoices FOR SELECT USING (tenant_id::TEXT
= current_setting('app.tenant', true));
SET app.tenant = 'tenant_a';
SELECT invoice_id, tenant_id, amount FROM invoices; -- only tenant_a
SET app.tenant = 'tenant_b';
SELECT invoice_id, tenant_id, amount FROM invoices; -- only tenant_b

```

-----Completion of Demo

-----Only do this if NOT on a Windows laptop, 😊

F) pgvector (AI embeddings + ANN) – requires pgvector installed on cluster.

Tip: This will fail if Stackbase won't install pgvector on Windows. Best done on a docker image or other demo.

```

DO $$ BEGIN CREATE EXTENSION IF NOT EXISTS pgvector; END $$;
DROP TABLE IF EXISTS documents CASCADE;
CREATE TABLE documents (
    doc_id SERIAL PRIMARY KEY,
    content TEXT NOT NULL,
    embedding VECTOR(3)
);
INSERT INTO documents (content, embedding) VALUES
('How to vacuum analyze tables', '[0.10, 0.05, 0.90]'),

```

```

('Spatial joins with PostGIS', '[0.20, 0.80, 0.10]'),
('Parallel aggregates in PG', '[0.15, 0.10, 0.85]'),
('Managing partitions well', '[0.12, 0.12, 0.88]');
CREATE INDEX IF NOT EXISTS documents_embedding_ivf
  ON documents USING ivfflat (embedding vector_cosine_ops) WITH (lists = 4);
ANALYZE documents;
EXPLAIN (ANALYZE, BUFFERS)
SELECT doc_id, content, 1 - (embedding <=> '[0.12, 0.10, 0.88]') AS
cosine_similarity
FROM documents
ORDER BY embedding <-> '[0.12, 0.10, 0.88]'
LIMIT 3;

```

-----Completion of Demo

-----Offer PDF of demo to anyone who would like it

Troubleshooting Notes

- **No parallel workers?** Check `max_parallel_workers` and `max_worker_processes`, and ensure table size/queries are large enough to trigger parallelism.
- **FDW connection fails?** Confirm host/port, local trust auth, or configure a proper `CREATE USER MAPPING ... OPTIONS (user '...', password '...')`.
- **pgvector not found?** Install the extension package for your distro/cloud or enable it in managed service settings.
- **Poor ANN recall?** Increase IVFFLAT lists; for high-dim embeddings, also tune probes at query time: `SET ivfflat.probes = 8;`
- **Partition pruning not happening?** Ensure constraints are declarative partitions and your predicates are sargable (use `>=`/`<` on the partition key).

Cheat Sheet

Session Flags - \timing on measure query latency - `SET max_parallel_workers_per_gather = 4;` -- enable parallelism per gather

JSONB - Index: `CREATE INDEX ... USING GIN (profile jsonb_path_ops);` - Contains: `WHERE profile @> '{"prefs":{"lang":"en"}}'` - Key text: `profile ->> 'tier'`

FDW - Enable: `CREATE EXTENSION postgres_fdw;` - Server: `CREATE SERVER ... FOREIGN DATA WRAPPER postgres_fdw OPTIONS (...);` - Map: `CREATE USER MAPPING FOR CURRENT_USER SERVER ... OPTIONS (...);` - Foreign table: `CREATE FOREIGN TABLE ... SERVER ... OPTIONS (table_name '...');`

Partitioning - Create: ... PARTITION BY RANGE (order_date); - Partition: FOR VALUES FROM ('2025-02-01') TO ('2025-03-01'); - Prune demo: add WHERE order_date >= '2025-02-01' AND order_date < '2025-03-01'

Parallel - Look for Gather in EXPLAIN (ANALYZE, VERBOSE)

RLS - Enable: ALTER TABLE t ENABLE ROW LEVEL SECURITY; - Policy: CREATE POLICY p ON t USING (tenant_id = current_setting('app.tenant', true)); - Switch: SET app.tenant = 'tenant_a';

pgvector - Enable: CREATE EXTENSION vector; - Table: embedding VECTOR(768) (or your dimension) - Index: USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100); - Query: ORDER BY embedding <-> :query LIMIT 5;

Handy psql - \d+ table , describe table - \dx , list extensions - \watch 1 , rerun last query every second

Cleanup (optional)

-- Remove FDW objects

DROP FOREIGN TABLE IF EXISTS ext_products;

DROP SERVER IF EXISTS ext_source_svr **CASCADE**;

-- Drop demo DBs (run from another DB)

-- DROP DATABASE pg_top10_demo WITH (FORCE);

-- DROP DATABASE ext_source WITH (FORCE);

End of DEMO